```
public record UserModel                ❷
{
    [Required]
    [StringLength(100)]
    [Display(Name = "Your name")]
    public string Name { get; set; }

    [Required]
    [EmailAddress]
    public string Email { get; set; }
}
```

❶ Adds the validation filter to the endpoint

❷ The UserModel defines its validation requirements using DataAnnotations attributes.

Listing 7.12 shows how you can validate a complex type, but in some cases, you may want to validate simple types. You may want to validate that the `id` value in the following handler should be between `1` and `100`:

```
app.MapGet("/user/{id}", (int id) => $"Received {id}")
    .WithParameterValidation();
```

Unfortunately, that's not easy to do with `DataAnnotations` attributes. The validation filter will check the `int` type, see that it's not a type that has any `DataAnnotations` on its properties, and won't validate it.

WARNING Adding attributes to the handler, as in `([Range(1, 100)] int id)`, doesn't work. The attributes here are added to the *parameter*, not to properties of the `int` type, so the validator won't find them.

There are several ways around this problem, but the simplest is to use the `[AsParameters]` attribute you saw in section 7.9 and apply annotations to the model. The following listing shows how.